
FoodLoop

Buenas Prácticas, Normas y Estándares del Proyecto

Matriz de Roles y Responsabilidades

Universidad Tecnológica de Xicotepec de Juárez (UTXJ)
Ingeniería en Desarrollo y Gestión de Software — Grupo 9B

1. Introducción

Este documento define las buenas prácticas, normas y estándares que rigen el desarrollo del proyecto FoodLoop, así como la matriz de roles y responsabilidades del equipo. El objetivo es asegurar consistencia en el trabajo colaborativo, facilitar la incorporación de nuevas funcionalidades y mantener trazabilidad sobre quién es responsable de cada parte del sistema.

2. Buenas Prácticas, Normas y Estándares del Proyecto

2.1 Estándares de código

- Nomenclatura en inglés para variables, funciones y componentes; nombres descriptivos que expliquen su propósito.
- Componentes de React en PascalCase; funciones y variables en camelCase; constantes globales en UPPER_SNAKE_CASE.
- Un componente/función debe tener una única responsabilidad; evitar archivos con más de una responsabilidad mezclada.
- Comentarios solo donde la lógica no sea evidente por sí misma; evitar comentarios redundantes.
- Manejo de errores explícito (try/catch) en toda llamada a servicios externos (IA, base de datos, APIs de terceros).

2.2 Control de versiones (Git Flow)

- Rama main reservada para versiones estables y entregables; nunca se trabaja directo sobre ella.
- Rama develop como integración de las funcionalidades ya probadas.
- Una rama de feature por tarea, con prefijo según el tipo: feature/, fix/, docs/.
- Mensajes de commit cortos y en modo imperativo, describiendo el cambio (ej. "agrega validación de login").
- Todo Pull Request requiere revisión de al menos un integrante distinto al autor antes del merge.
- No se suben archivos de configuración con credenciales (.env) ni carpetas de dependencias (node_modules).

2.3 Documentación

- El README del repositorio se mantiene como carta de presentación del proyecto: breve, sin detalle técnico exhaustivo.
- La documentación técnica a profundidad (arquitectura, base de datos, API) vive en documentos independientes dentro de la carpeta de Documentación.
- Todo cambio relevante de alcance, tecnología o proceso se documenta antes de implementarse, no después.
- Los reportes de avance y bitácoras se actualizan por cada periodo/entrega, no al final del proyecto.

2.4 Seguridad

- Las claves y secretos (API keys, JWT secret, credenciales SMTP) se manejan siempre mediante variables de entorno (.env), nunca en el código fuente.
- Las contraseñas de usuario se almacenan siempre con hashing, nunca en texto plano.
- Toda ruta que exponga datos de usuario requiere autenticación mediante middleware validado.

2.5 Comunicación y colaboración

- Actualizaciones de estado diarias por WhatsApp; bloqueos críticos se comunican de inmediato al equipo.
- Sincronización técnica semanal por videollamada (Google Meet) para revisar avances y resolver bloqueos.
- Se respetan los tiempos de trabajo enfocado de cada integrante; el contacto inmediato se reserva para bloqueos críticos.

2.6 Calidad y revisión

- Ninguna funcionalidad se integra a develop sin al menos una revisión de código por otro integrante.
- Antes de cada entrega se verifica que el flujo principal (registro, login, funcionalidad nueva) funcione de extremo a extremo.
- Los cambios que afecten a más de un módulo se prueban de forma integrada, no solo de forma aislada.
- Toda funcionalidad nueva se contrasta contra los objetivos específicos del proyecto antes de aceptarse, para no desviar el alcance.

3. Matriz de Roles y Responsabilidades

La siguiente tabla resume el rol principal, las responsabilidades y los entregables asociados a cada integrante del equipo.

Integrante	Rol	Responsabilidades principales	Entregables asociados
Josué Atlai Martínez Otero	Líder de Frontend — Web y Móvil	Diseño de wireframes y prototipo en Figma; maquetación responsiva en React (web) y en el cliente móvil; navegación entre pantallas; consumo de la API REST del backend.	Interfaz web, interfaz móvil, prototipo de alta fidelidad en Figma
José Agustín Jiménez Castillo	Desarrollador — Smartwatch	Desarrollo del cliente para smartwatch; consumo del payload ligero de presupuesto/lista de compra; implementación de la guía de receta paso a paso en el reloj; documentación funcional y de soporte.	Cliente smartwatch, documentación de soporte
Christian Paúl Rodríguez Pérez	Líder de Backend e IA	Diseño y mantenimiento de la API (Node.js/Express); integración con los proveedores de IA generativa (Gemini/OpenAI); lógica de negocio del sistema.	API REST, servicios de integración con IA, documentación técnica del backend
Jonathan Baldemar Ramírez Reyes	Líder de Documentación y Base de Datos	Diseño del modelo de datos; documentación técnica y de gestión del proyecto (README, planes, reportes); consolidación de entregables del equipo.	Documentación del proyecto, modelo de datos, reportes de avance

3.1 Matriz de responsabilidades por actividad (RACI simplificado)

Detalle de quién ejecuta, quién colabora y quién debe mantenerse informado en cada actividad principal del proyecto.

Actividad / Entregable	Josué	Agustín	Christian	Jonathan
Interfaz web (React)	R	I	C	I
Interfaz móvil	R	I	C	I
Cliente smartwatch	I	R	C	I
API / lógica de backend	C	C	R	I
Integración con IA (Gemini/OpenAI)	I	I	R	I
Modelo de base de datos	I	I	C	R
Documentación del proyecto (README, planes, reportes)	C	C	C	R
Control de versiones y revisión de Pull Requests	C	C	C	C

R = Responsable directo **C** = Consultado / colabora **I** = Informado sobre el avance